In [6]:

```
class Book:
    def __init__(self, no, a_name, publisher):
        self.no = no
        self.a_name = a_name
        self.publisher = publisher

    def display(self):
        print("No. of author:", self.no)
        print("Name of author:", self.a_name)
        print("Name of publisher:", self.publisher)

class CourseBook(Book):
    def __init__(self, no, a_name, publisher, cname):
        super().__init__(no, a_name, publisher)
        self.cname = cname

    def display(self):
        super().display()
        print("Name of course:", self.cname)


b = CourseBook(1, "Aisha", "Alondra", "Python")
c.display()
print()
b.display()


```

```
No. of author: 1
Name of author: Aisha
Name of publisher: Alondra

No. of author: 1
Name of author: Aisha
Name of publisher: Alondra
Name of course: Python
```

2. implement following : create class "staf' take name & salary as its data member. the derived class "teching" has subject as data member and class "non_teching" has department has its data member. the derived class method overrids the methods of the base class

In [10]:

```
class Staf:
    def __init__(self,name,salary):
        self.name=name
        self.salary=salary

    def display(self):
        print("Name is :",self.name)
        print("Salary is :",self.salary,"INR")

class Teach(Staf):
    def __init__(self,name,salary,subject):
        super().__init__(name,salary)
        self.subject=subject

    def display(self):
        super().display()
        print("Subject is :",self.subject)

class NonTech(Staf):
    def __init__(self,name,salary,dept):
        super().__init__(name,salary)
        self.dept=dept

    def display(self):
        super().display()
        print("Department is :",self.dept)


t=Teach("A",20000,"PS")
t.display()
print()
n=NonTech("B",40000,"director")
n.display()
```

```
Name is : A
Salary is : 20000 INR
Subject is : PS

Name is : B
Salary is : 40000 INR
Department is : director
```

3. Create a class called "Student" having name and email as its data members. and constructor and put data as bound method. the init function should assign the values passed as perameter to the variable. the put data function should display the data of student. Create another class classed "PHDGuide" having name and email as its data member you can use students as a variable as a blank list. the students variable is the list of student under the guide the "PHDGuide" class have four bound methods. init, putdata ,add ,remove. init method initialize variable , put data shows the data of the guide including the list of student , the add method should add a student to the list of students of the guide. and remove function should remove the student if student exist in the list of students.

In [44]:

```
class Student:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    def putdata(self):
        print("Student name:", self.name)
        print("Student email:", self.email)

class PHDGuide:
    def __init__(self, name, email):
        self.name = name
        self.email = email
        self.s = []

    def putdata(self):
        print("Guide name:", self.name)
        print("Guide email:", self.email)



    def add(self, s):
        self.s.append(s)
        print(f"Student {s.name} added to the guide {self.name}'s list.

    def remove(self, s):
        if s in self.s:
            self.s.remove(s)
            print(f"Student {s.name} removed from the guide {self.name}
        else:
            print(f"Student {s.name} is not in the guide {self.name}'s



a=Student("aisha","aisha123@gmail.com")
b=Student("aahana","ahanaslife@gmail.com")
c=Student("lala","lala@gmail.com")
a.putdata()
print()
b.putdata()
print()
g1=PHDGuide("Prince Patel","789Prince@gmail.com")
g1.add(a)
g1.add(b)
print()
g1.putdata()
g1.remove(c)

```

4. Create 3 class employes , works and net_salary . name employe class as an abstract class . employee class should have method for the following task.

1. to get emp details emp id ,name and salary from user
2. to print employe details.
3. to return salary.
4. an abstract method emp_id.

Now perks class calculate the

1. DA,HRF,PF.
2. to print individual and total perks

Net_salary

1. calculate total salary after perks
2. print the employe details with da,hra,pf,net saraly

da=35% hra=17% pf=12%

In [67]:

```
from abc import ABC,abstractmethod
class emp(ABC):
    def __init__(self,id,name,salary):
        self.id = None
        self.name = None
        self.salary = None
    def details(self):
        self.id=input("enter your id:")
        self.name=input("enter your name:")
        self.salary=int(input("enter tour salary:"))
        print()

    def display(self):
        print("------------------information of employee---------------
        print("id :",self.id)
        print("name: ",self.name)
        print("salary: ",self.salary)

    def slar(self):
        return self.salary

    @abstractmethod
    def empid(self):
        pass

class Pearks(emp):
    def __init__(self):
        super().__init__()
        self.da=0
        self.hra=0
        self.pf=0
        self.total_pearks=0
    def calculate_perks(self):
        self.da = self.salary * 0.35
        self.hra = self.salary * 0.17
        self.pf = self.salary * 0.12
        self.total_perks = self.da + self.hra + self.pf

    def print_perks(self):
        print("DA:", self.da)
        print("HRA:", self.hra)
        print("PF:", self.pf)
        print("Total Perks:", self.total_perks)


class NetSalary(Pearks):
    def calculate_net_salary(self):
        self.net_salary = self.salary + self.da + self.hra - self.pf

    def print_net_salary(self):
        self.print_emp_details()
        self.print_perks()
        print("Net Salary:", self.net_salary)

    def get_emp_id(self):
        return self.emp_id

```

```
c=NetSalary()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-67-019eef934228> in <module>
     56         return self.emp_id
     57
---> 58 c=NetSalary()

TypeError: Can't instantiate abstract class NetSalary with abstract methods empid
```

In [ ]:

```

```